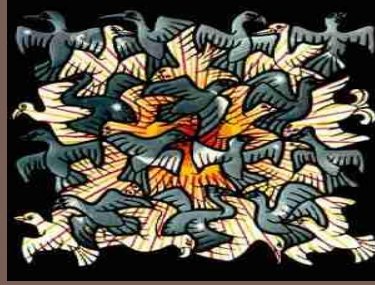


ARQUITETURA E PADRÕES DE SOFTWARE

Prof. Luciano Vale, Msc
lucianovale@unifor.br



PADRÕES GOF — PARTE II

Padrões GoF – parte I

3

- ❑ Singleton
- ❑ Composite
- ❑ Façade
- ❑ Adapter
- ❑ Template Method
- ❑ Strategy
- ❑ Iterator

■	Padrões de Criação
■	Padrões Estruturais
■	Padrões de Comportamento

Padrões GoF – parte II

4

- Prototype
- Decorator
- State
- Observer
- Memento
- Method
- Abstract Factory



■	Padrões de Criação
■	Padrões Estruturais
■	Padrões de Comportamento

5

Prototype

Prototype

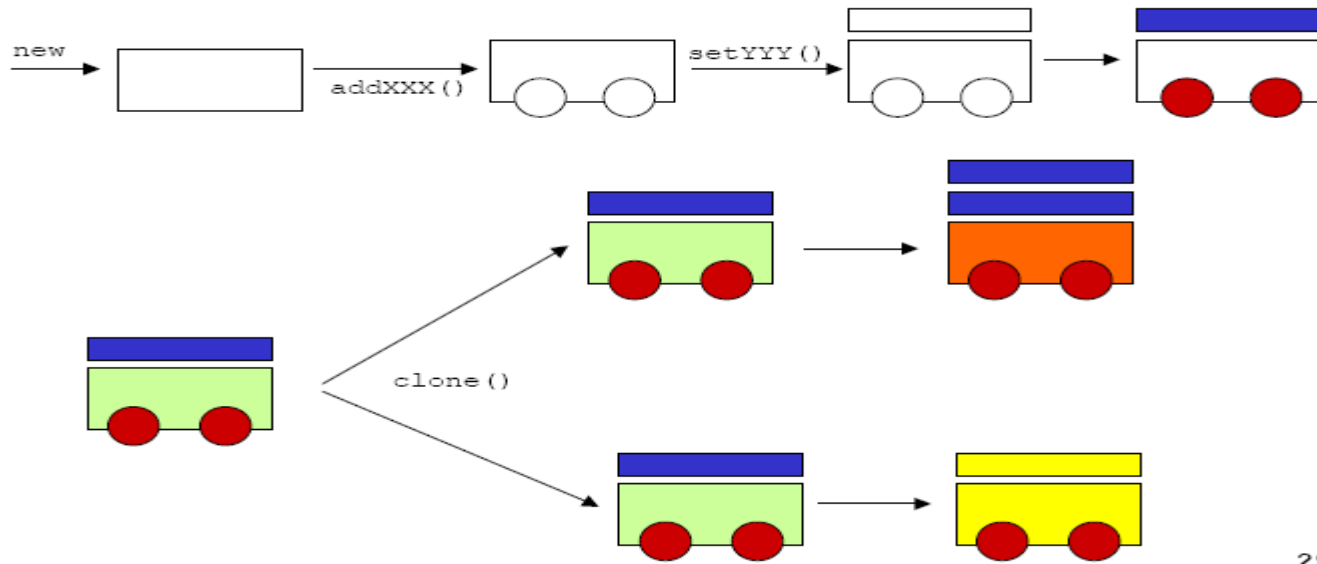
6

- **Intenção:** Permitir que um cliente crie objetos por meio de operações de cópia de uma instância prototípica.
 - ▣ GoF Book: *“Specify the kinds of objects to create using a prototypical instance, and create new objects by copying this prototype.”*
- **Vantagem:**
 - ▣ Reutilizar o estado do protótipo: o objeto criado é um **clone** do objeto original.
 - ▣ Em vez de criar novas instâncias do zero, uma aplicação pode criar cópias de instâncias pré-existentes (*protótipos*) e modificar o estado dessas cópias, conforme a necessidade.

Prototype

7

□ Motivação



20

Prototype – cópias rasa e profunda

8

- Há dois tipos de cópia que podem ser aplicados a um objeto.
 - ▣ Na **cópia rasa** (*shallow copy*), o objeto é clonado, **mas seus componentes são referenciados**.
 - ▣ Na **cópia profunda** (*deep copy*), o objeto e seus componentes são clonados.

Prototype – cópias rasa e profunda

9

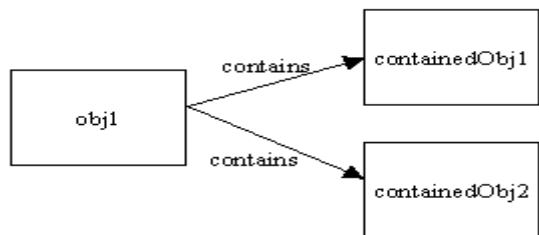


Figura 1. O estado original do obj1

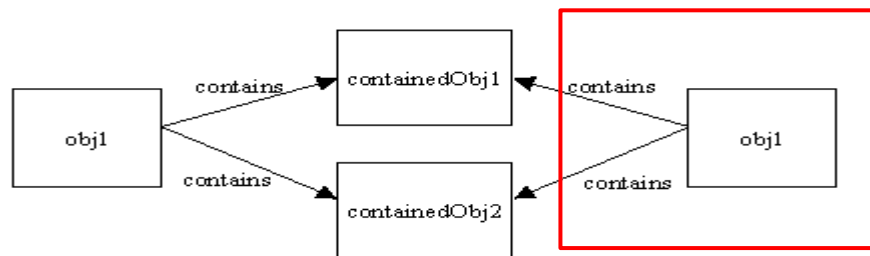


Figura 2. Após o shallow copy do obj1

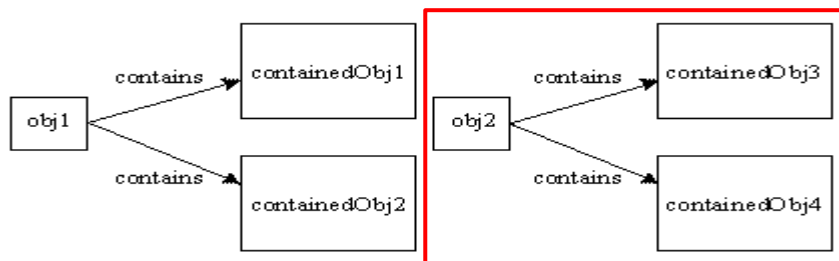
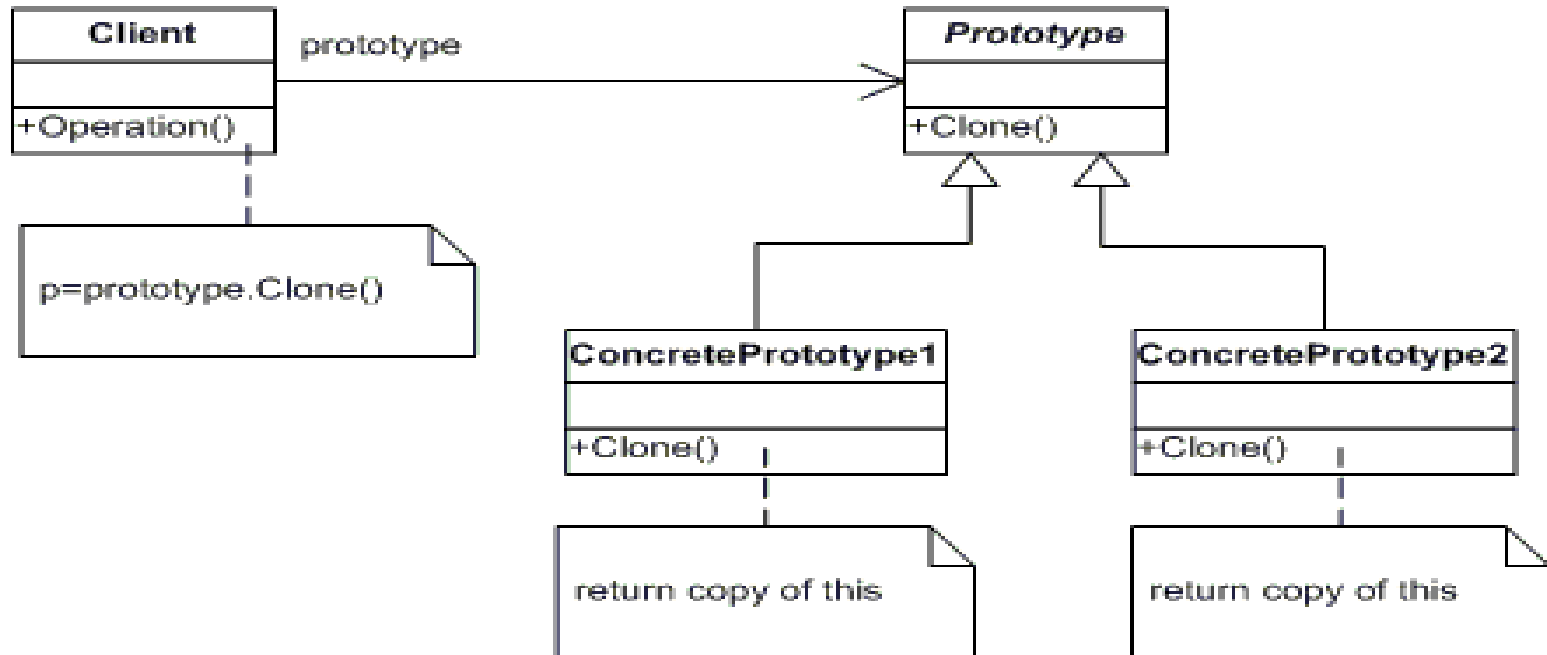


Figura 3. Após o deep copy do obj1

Prototype

10

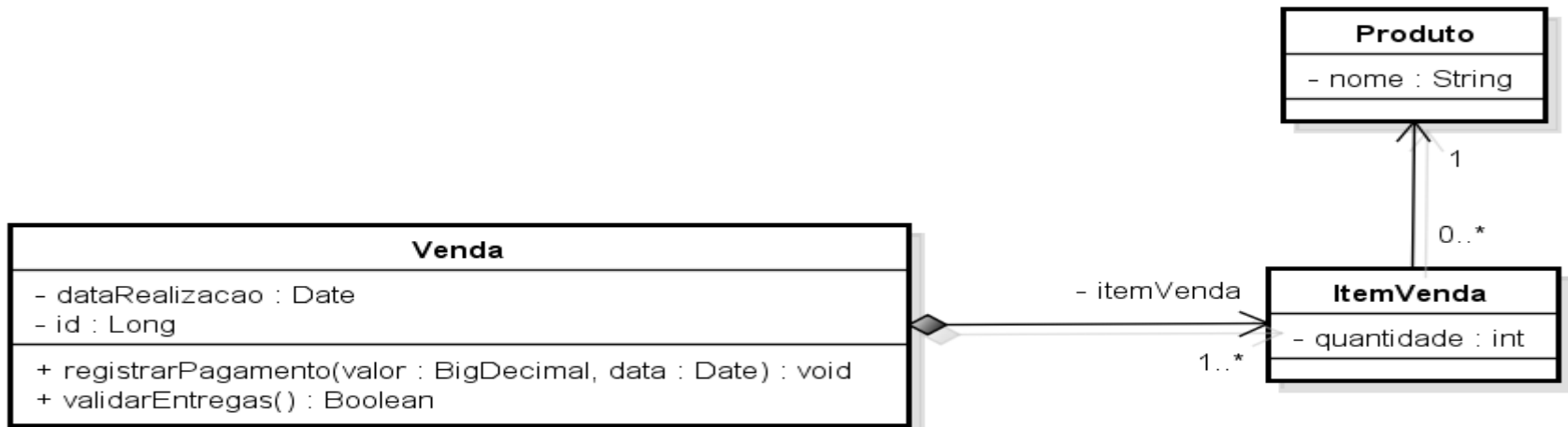
□ Estrutura



Prototype - exemplo

11

- “Deve ser possível registrar uma nova venda a partir da cópia de uma venda pré-existente.”



Prototype – exemplo (cont.)

12

A interface Cloneable do Java é uma interface que marca, não tem nenhum método. O código a seguir mostra um simples processo de clonagem. A interface Cloneable não tem nenhum membro também, e é usada para indicar que a classe (que implementa a interface) permite uma clonagem bit a bit de um objeto, processo esse simplesmente chamado de "clonagem". Uma exceção conhecida como "CloneNotSupportedException" é mostrada se a função clone() é chamada numa classe em que não se implementou a interface Cloneable. No processo de clonagem o construtor do objeto não é chamado. Então, clonar pode ser definido como uma exata cópia do objeto original.

```
public class Venda implements Cloneable {
```

```
    public Venda clone() throws CloneNotSupportedException {  
        Venda outraVenda = new Venda();  
        outraVenda.dataRealizacao = this.dataRealizacao;  
        for (ItemVenda item : itensVenda) {  
            outraVenda.com(item.clone());  
        }  
        return outraVenda;  
    }
```

```
public class ItemVenda implements Cloneable {
```

```
    protected ItemVenda clone() throws CloneNotSupportedException {  
        return new ItemVenda(this.produto, this.quantidade);  
    }
```

Prototype – exemplo (cont.)

13

```
public class VendaService {  
    private ClienteService clienteService = new ClienteService();  
    private VendaRepositorio vendaRepositorio = RepositorioFabrica  
        .getVendaRepositorio();  
  
    public boolean excluirVenda(Long id) {  
  
    public boolean incluirVenda(Venda venda) {  
  
    public List<Venda> obterVendas() {  
  
    public Venda iniciarVenda(Long idCliente, Date dataRealizacao) {  
  
    public void registrarPagamentoVenda(Long idVenda, Date dataRealizacao,  
  
    public Venda duplicarVenda(Long idVenda) throws CloneNotSupportedException {  
        Venda venda = vendaRepositorio.obterPorId(idVenda);  
        return venda.clone();  
    }  
}
```

14

Decorator

Decorator

15

- Intenção: Anexar responsabilidades adicionais a um objeto dinamicamente. Decoradores fornecem uma alternativa flexível ao uso de herança para adição de funcionalidades extensoras.
- ▣ GoF Book: *“Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.”*

Decorator – herança vs composição

16

- Podemos usar duas alternativas para acrescentar responsabilidades a objetos:
 1. Herança
 - Explosão de classes
 - Inflexibilidade – herança é um relacionamento estático.
 2. Decorator
 - Adiciona ou remove **dinamicamente** responsabilidades em um objeto. Usa o **OCP (Open-Closed Principle – SOLID)**.

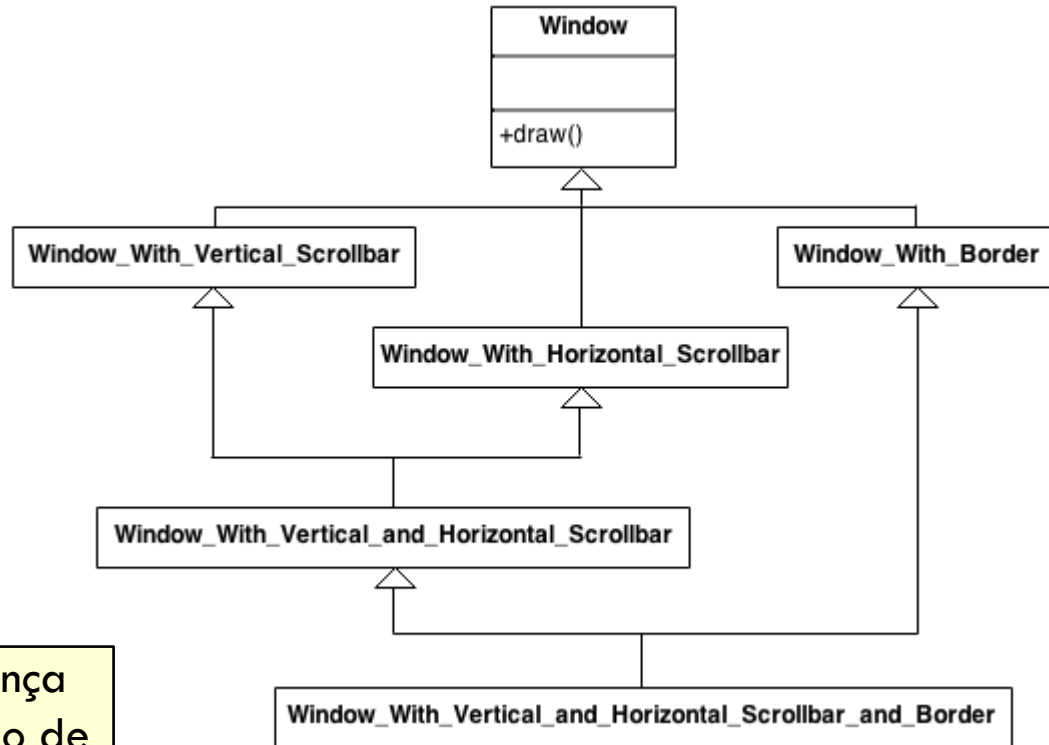
Decorator - motivação

17

- Suponha que você esteja trabalhando em um kit de ferramentas de interface de usuário e deseje dar suporte à adição de bordas e barras de rolagem às janelas.
- Você poderia definir uma hierarquia de herança...

Decorator - motivação

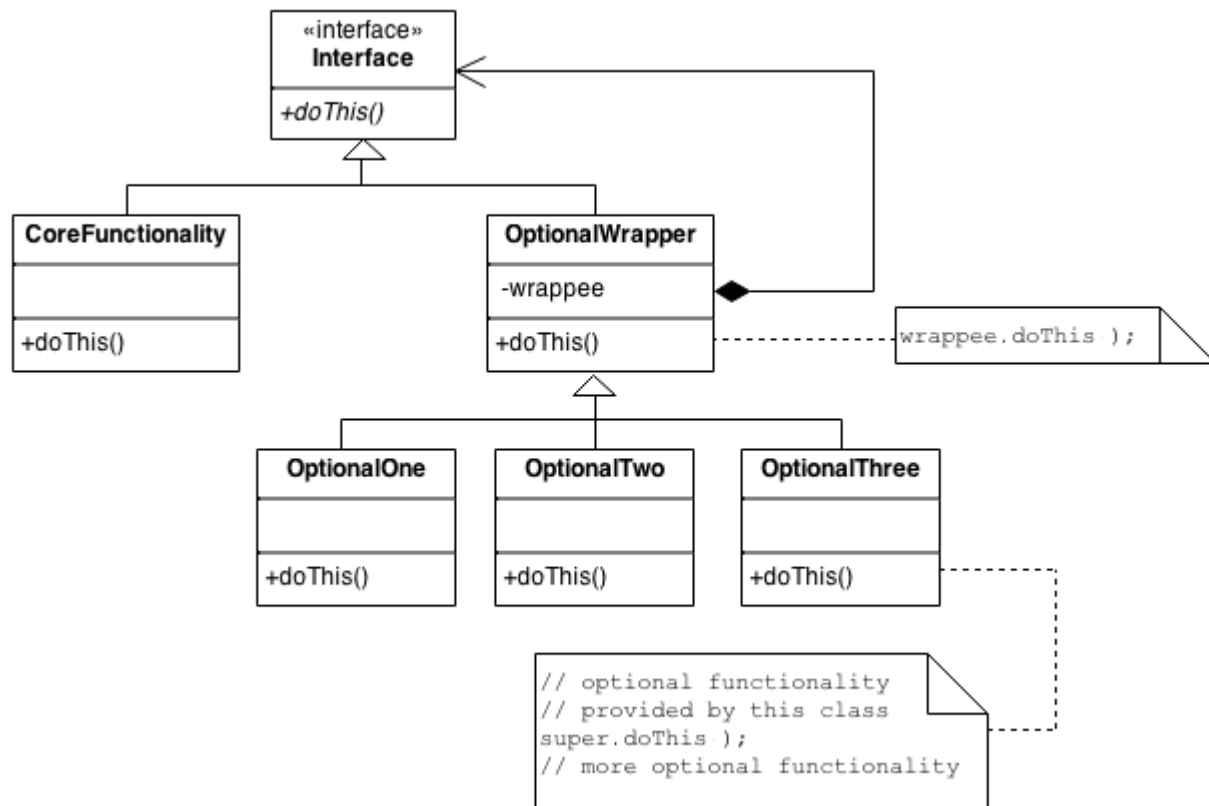
18



Mas o uso de herança
leva a uma explosão de
quantidade de classes

Decorator - estrutura

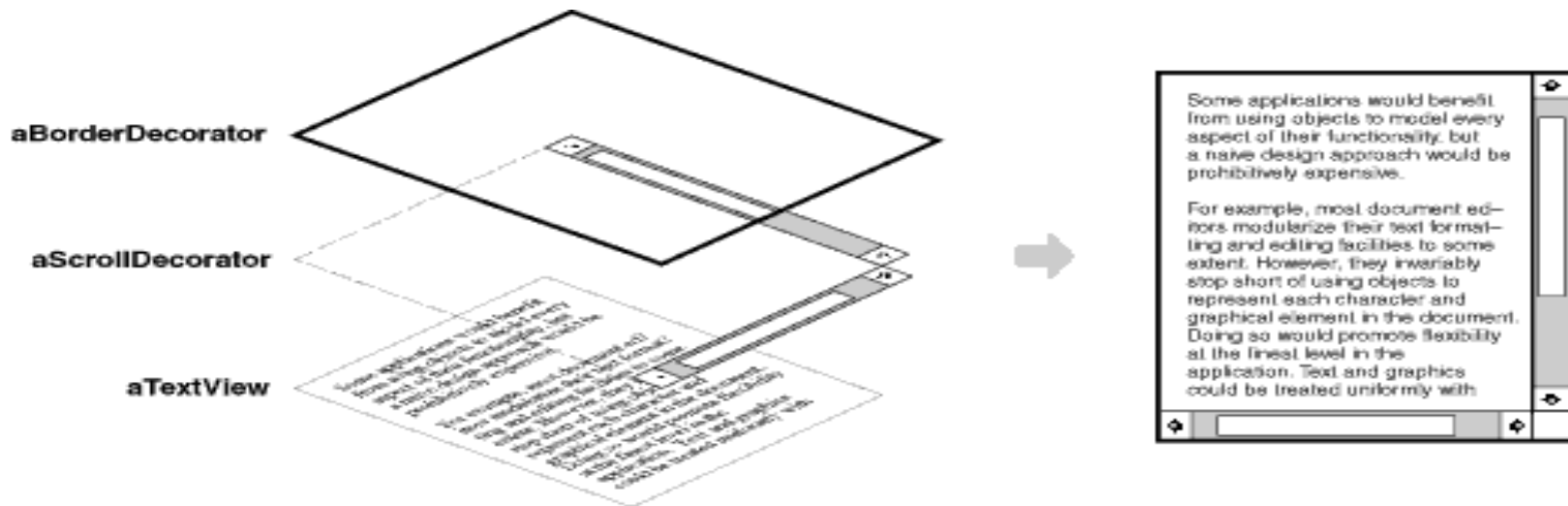
19



Decorator - exemplo 01

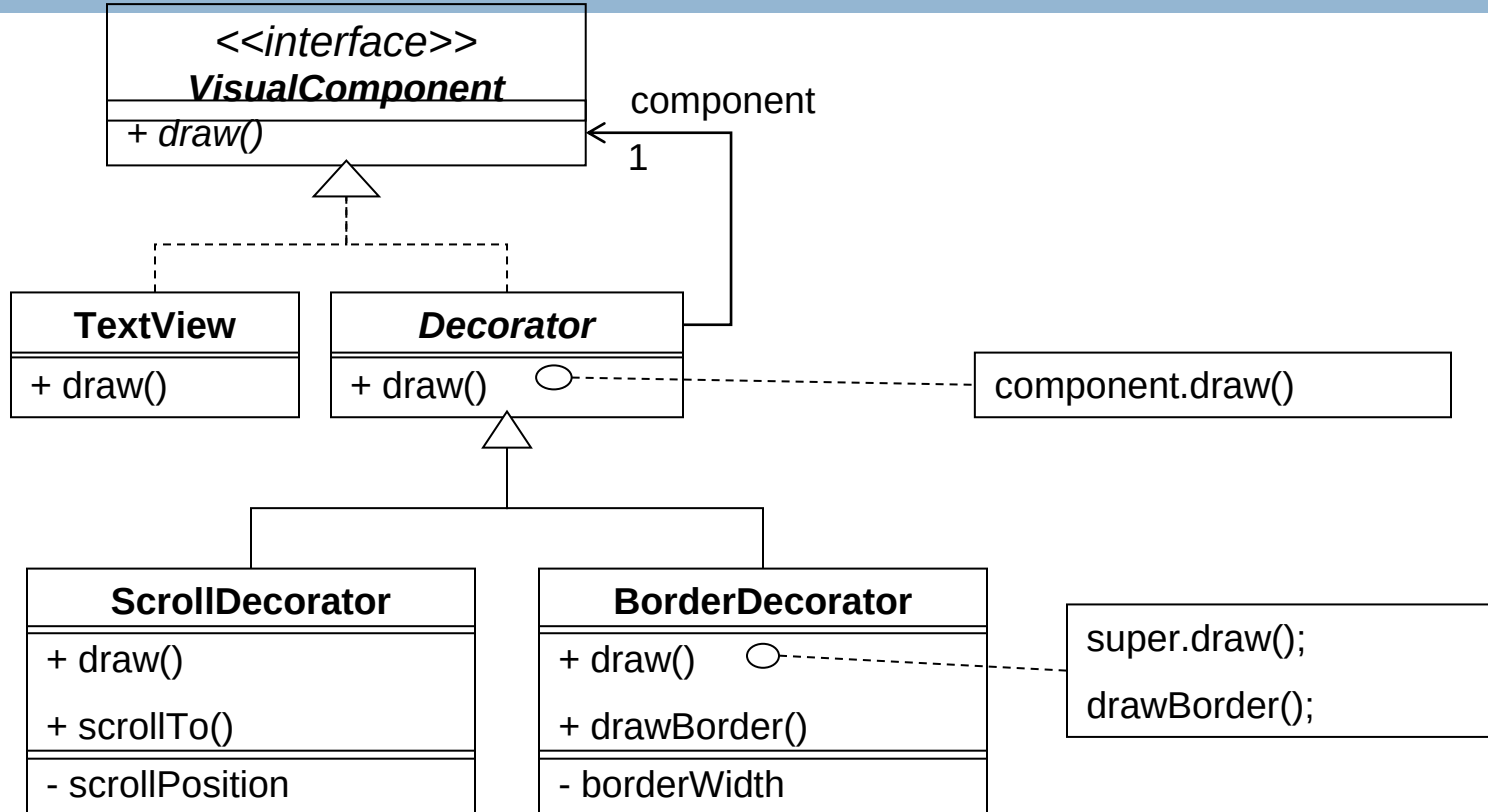
20

- Componentes de interface gráfica (e.g., caixas de texto) **decoradas** com barras de rolagem e bordas.



Decorator - exemplo 01

21



Decorator - exemplo 01

22

```
public interface VisualComponent {  
    public void draw();  
}
```

```
3 public class TextView implements VisualComponent {  
4  
5     public TextView(int height, int width) {  
6         // ...  
7     }  
8  
9     public void draw() {  
10        System.out.println("Draws the text view.");  
11    }  
12  
13 }
```

```
3 public abstract class Decorator implements VisualComponent {  
4     VisualComponent component;  
5 }
```

Decorator - exemplo 01

23

```
3 public abstract class Decorator implements VisualComponent {  
4     VisualComponent component;  
5 }
```

```
3 public class HorizontalScrollBarDecorator extends Decorator {  
4  
5     public HorizontalScrollBarDecorator(VisualComponent component) {  
6         this.component = component;  
7     }  
8  
9     public void draw() {  
10         super.draw();  
11         drawHorizontalScrollBar();  
12     }  
13  
14     private void drawHorizontalScrollBar() {  
15         System.out.println("Draws the horizontal scroll bar.");  
16     }  
17  
18 }
```

Decorator - exemplo 01

24

```
3 public abstract class Decorator implements VisualComponent {  
4     VisualComponent component;  
5 }
```

```
3 public class VerticalScrollBarDecorator extends Decorator {  
4  
5     public VerticalScrollBarDecorator(VisualComponent component) {  
6         this.component = component;  
7     }  
8  
9     @Override  
10    public void draw() {  
11        super.draw();  
12        drawVerticalScrollBar();  
13    }  
14  
15    private void drawVerticalScrollBar() {  
16        System.out.println("Draws the vertical scroll bar.");  
17    }  
18  
19 }
```

Decorator - exemplo 01

25

```
3 public abstract class Decorator implements VisualComponent {
4     VisualComponent component;
5 }
```

```
3 public class BorderDecorator extends Decorator {
4
5     public BorderDecorator(VisualComponent component) {
6         this.component = component;
7     }
8
9     @Override
10    public void draw() {
11        super.draw();
12        drawBorder();
13    }
14
15    private void drawBorder() {
16        System.out.println("Draws the border.");
17    }
18
19 }
```

Decorator - exemplo 01

26

```
3 public class Principal {  
4  
5     public static void main(String[] args) {  
6         VisualComponent w = new TextView(80, 24);  
7         VisualComponent d1 = new VerticalScrollBarDecorator(w);  
8         VisualComponent d2 = new HorizontalScrollBarDecorator(d1);  
9         VisualComponent d3 = new BorderDecorator(d2);  
10  
11         d3.draw();  
12     }  
13 }  
14 }  
15 }
```

Decorator - exemplo 02 - Java

27

```
// a caixa de texto é o componente "decorado"
```

```
JTextArea txt = new JTextArea();
```

```
// "decora" a caixa de texto com barras de rolagem
```

```
Component comp = new JScrollPane(txt);
```

```
// adiciona a caixa de texto decorada com barras de rolagem no form
```

```
getContentPane().add(comp);
```

```
Widget aWidget = new BorderDecorator(  
    new HorizontalScrollBarDecorator(  
        new VerticalScrollBarDecorator(  
            new Window( 80, 24 ) ) ) ) );
```

```
aWidget.draw();
```

```
Stream aStream = new CompressingStream(  
    new ASCII7Stream( new FileStream( "fileName.dat" ) ) );
```

```
aStream.putString( "Hello world" );
```

28

State

Estado de um objeto

29

- ❑ Objeto (instância): entidade que possui estado, comportamento e identidade.
- ❑ A estrutura e o comportamento de objetos semelhantes são definidos em sua classe (tipo).
- ❑ O **estado** de um objeto engloba suas propriedades (geralmente estáticas) mais os valores atuais (geralmente dinâmicos) atribuídos a cada uma dessas propriedades.

State

30

- Intenção: permitir a um objeto alterar seu comportamento em função de uma mudança no seu estado. O objeto parecerá ter mudado sua classe.
 - ▣ GoF Book: “Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.”
- Move os comportamentos dependentes do estado do objeto original para uma série de outros objetos, um para cada estado.

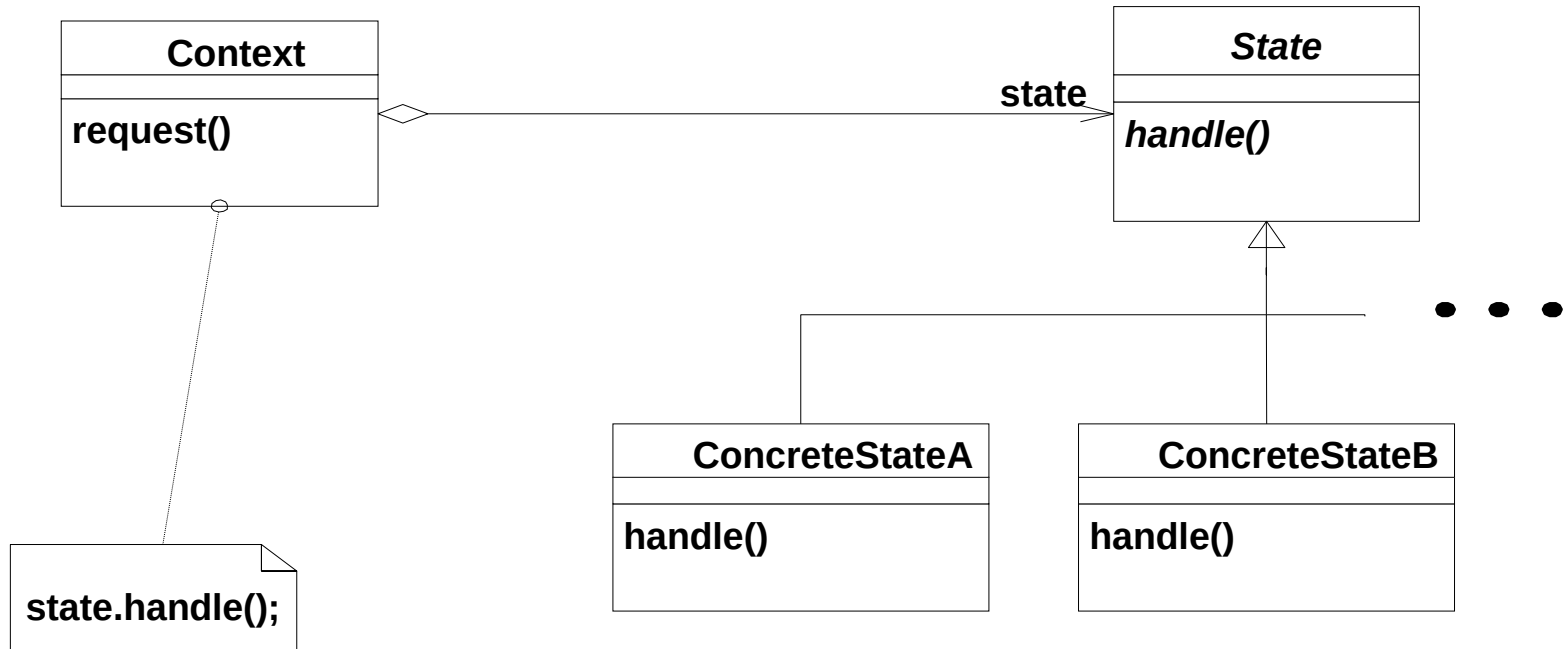
State - estrutura

31

- Uma hierarquia de classes representa os estados de um objeto.
- Cada subclasse da hierarquia
 - ▣ representa um dos possíveis estados do objeto;
 - ▣ implementa o comportamento correspondente a seu estado.
- Objeto é ligado a seu estado corrente por meio do relacionamento de composição.

State - estrutura

32



State – exemplo

33

- Uma locadora categoriza seus clientes em função dos pontos acumulados em locações de filmes.
- Ao alugar um filme, um cliente pode receber descontos diferentes, em função da categoria a que pertence:
 - MembroBronze (≤ 500) $\rightarrow$ desconto 0%, ganha 40 pts
 - MembroPrata (> 500 e ≤ 1000) $\rightarrow$ desconto 3%, ganha 60 pts
 - MembroOuro (> 1000 e ≤ 5000) $\rightarrow$ desconto 5%, ganha 80 pts
 - MembroPlatina (> 5000) $\rightarrow$ desconto 10%, ganha 100 pts.

State – exemplo (cont.)

Sem State

34

```
5 public class Cliente {
6
7+   public Cliente(String nome, int pontosIniciais) {
11
13+   * Registra aluguel de filme ao cliente, em certa data.
19+   public double registrarLocacao(Filme filme) {
26
27+   int getPontosGanhos() {
38
39+   double getDesconto() {
50
51+   public void usarPontos(int pontos) {
57
58+   public int getQuantidadePontos() {
51
52-   public void setQuantidadePontos(int quantidadePontos) {
53       this.quantidadePontos = quantidadePontos;
54   }
55
56+   public void addQuantidadePontos(int pontosGanhos) {
59
70+   private void registrar(Filme filme, Date dataLocacao) {
73
74+   public String getNome() {
77
78   private static final int USQ_MAX_PONTOS = 100;
79   private static final int PTS_BRONZE = 500;
80   private static final int PTS_PRATA = 1000;
81   private static final int PTS_OURO = 5000;
82
83   private int quantidadePontos;
84   private String nome;
85 }
```

State – exemplo (cont.)

Sem State

35

```
20  /**
21   * Registra aluguel de filme ao cliente, em certa data.
22   *
23   * @param filme
24   *       filme a ser alugado.
25   * @return o valor a pagar pela locação do filme.
26   */
27  public double registrarLocacao(Filme filme) {
28      registrar(filme, new Date());
29      double precoLocacao = filme.getPrecoLocacao() * (1 - getDesconto());
30      addQuantidadePontos(getPontosGanhos());
31      return precoLocacao;
32  }
```

```
26  int getPontosGanhos() {
27      if (quantidadePontos <= PTS_BRONZE) {
28          return 40;
29      } else if (quantidadePontos <= PTS_PRATA) {
30          return 60;
31      } else if (quantidadePontos <= PTS_OURO) {
32          return 80;
33      } else {
34          return 100;
35      }
36  }
```

```
38  double getDesconto() {
39      if (quantidadePontos <= PTS_BRONZE) {
40          return 0.00;
41      } else if (quantidadePontos <= PTS_PRATA) {
42          return 0.03;
43      } else if (quantidadePontos <= PTS_OURO) {
44          return 0.05;
45      } else {
46          return 0.10;
47      }
48  }
```

Os comportamentos de getPontosGanhos e getDesconto são **dependentes do estado**.

```
5 public class Cliente {
6     public Cliente(String nome, int pontosIniciais) {
7         this.nome = nome;
8         this.estadoAtual = new StateMembroBronze(this);
9         this.quantidadePontos = pontosIniciais;
10    }
11
12    private State getState() {
13        return estadoAtual;
14    }
15
16    void setState(State state) {
17        this.estadoAtual = state;
18    }
19
20    public void usarPontos(int pontos) {}
21
22    * Registra aluguel de filme ao cliente, em certa data.
23
24    public double registrarLocacao(Filme filme) {}
25
26    private double getDesconto() {}
27
28    private int getPontosGanhos() {}
29
30    public int getQuantidadePontos() {}
31
32    public void setQuantidadePontos(int quantidadePontos) {}
33
34    public void addQuantidadePontos(int pontosGanhos) {}
35
36    private void registrar(Filme filme, Date dataLocacao) {}
37
38    public String getNome() {}
39
40    private State estadoAtual;
41    private static final int USO_MAX_PONTOS = 100;
42    private int quantidadePontos;
43    private String nome;
44 }
```

State – exemplo (cont.)

Com State

37

```
27- /**
28   * Registra aluguel de filme ao cliente, em certa data.
29   *
30   * @param filme
31   *       filme a ser alugado.
32   * @return o valor a pagar pela locação do filme.
33   */
34- public double registrarLocacao(Filme filme) {
35     registrar(filme, new Date());
36     double precoLocacao = filme.getPrecoLocacao() * (1 - getDesconto());
37     addQuantidadePontos(getPontosGanhos());
38     return precoLocacao;
39
40 }
```

```
43- private double getDesconto() {
44     return getState().getDesconto();
45 }
46
47- private int getPontosGanhos() {
48     return getState().getPontosGanhos();
49 }
```

O comportamento **dependente do estado** é movido para as subclasses de State.

State – exemplo (cont.)

Com State

38

```
3 public abstract class State {  
4     protected Cliente cliente;  
5  
6     abstract double getDesconto();  
7  
8     abstract int getPontosGanhos();  
9  
10    abstract void realizarTransicao();  
11 }
```

State – exemplo (cont.)

Com State

39

```
3 public class StateMembroBronze extends State {
4     private static final int PTS = 40;
5     private static final int PTS_BRONZE = 500;
6
7     public StateMembroBronze(Cliente cliente) {
8         this.cliente = cliente;
9     }
10
11     public double getDesconto() {
12         return 0.0;
13     }
14
15     int getPontosGanhos() {
16         return PTS;
17     }
18
19     void realizarTransicao() {
20         if (cliente.getQuantidadePontos() > PTS_BRONZE) {
21             cliente.setState(new StateMembroPrata(cliente));
22         }
23     }
24 }
```

State – exemplo (cont.)

Com State

40

```
3 public class StateMembroOuro extends State {
4
5     final private int PTS = 80;
6
7     public StateMembroOuro(Cliente cliente) {
8         this.cliente = cliente;
9     }
10
11     public double getDesconto() {
12         return 0.05;
13     }
14
15     int getPontosGanhos() {
16         return PTS;
17     }
18
19     @Override
20     void realizarTransicao() {
21         if (cliente.getQuantidadePontos() > 5000) {
22             cliente.setState(new StateMembroPlatina(cliente));
23         } else if (cliente.getQuantidadePontos() <= 1000) {
24             cliente.setState(new StateMembroPrata(cliente));
25         }
26     }
27 }
```

State – exemplo (cont.)

Com State

41

```
3 public class StateMembroPlatina extends State {
4     final private int PTS = 100;
5
6     public StateMembroPlatina(Cliente cliente) {
7         this.cliente = cliente;
8     }
9
10    @Override
11    public double getDesconto() {
12        return 0.10;
13    }
14
15    @Override
16    int getPontosGanhos() {
17        return PTS;
18    }
19
20    @Override
21    void realizarTransicao() {
22        if (cliente.getQuantidadePontos() <= 5000) {
23            cliente.setState(new StateMembroOuro(cliente));
24        }
25    }
26 }
```

State – exemplo (cont.)

Com State

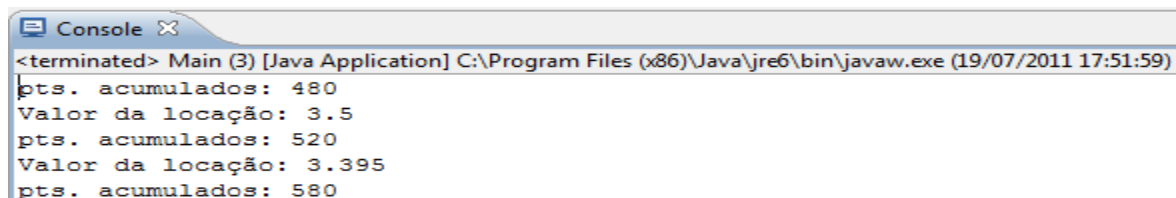
42

```
3 public class StateMembroPrata extends State {
4     final private int PTS = 60;
5
6     public StateMembroPrata(Cliente cliente) {
7         this.cliente = cliente;
8     }
9
10    @Override
11    public double getDesconto() {
12        return 0.03;
13    }
14
15    @Override
16    int getPontosGanhos() {
17        return PTS;
18    }
19
20    @Override
21    void realizarTransicao() {
22        if (cliente.getQuantidadePontos() > 1000) {
23            cliente.setState(new StateMembroOuro(cliente));
24        } else if (cliente.getQuantidadePontos() <= 500) {
25            cliente.setState(new StateMembroBronze(cliente));
26        }
27    }
28 }
```

State – exemplo (cont.)

43

```
3 public class Main {
4
5     public static void main(String[] args) {
6         Cliente cliente = new Cliente("Eduardo Bezerra", 480);
7
8         Filme umFilme = new Filme("Ben-Hur", 3.5);
9         Filme outroFilme = new Filme("The Wall", 3.5);
10
11         System.out.println("pts. acumulados: " + cliente.getQuantidadePontos());
12
13         System.out.println("Valor da locação: "
14             + cliente.registrarLocacao(umFilme));
15
16         System.out.println("pts. acumulados: " + cliente.getQuantidadePontos());
17
18         System.out.println("Valor da locação: "
19             + cliente.registrarLocacao(outroFilme));
20
21         System.out.println("pts. acumulados: " + cliente.getQuantidadePontos());
22     }
23 }
```



Console X

<terminated> Main (3) [Java Application] C:\Program Files (x86)\Java\jre6\bin\javaw.exe (19/07/2011 17:51:59)

pts. acumulados: 480
Valor da locação: 3.5
pts. acumulados: 520
Valor da locação: 3.395
pts. acumulados: 580

44

Observer

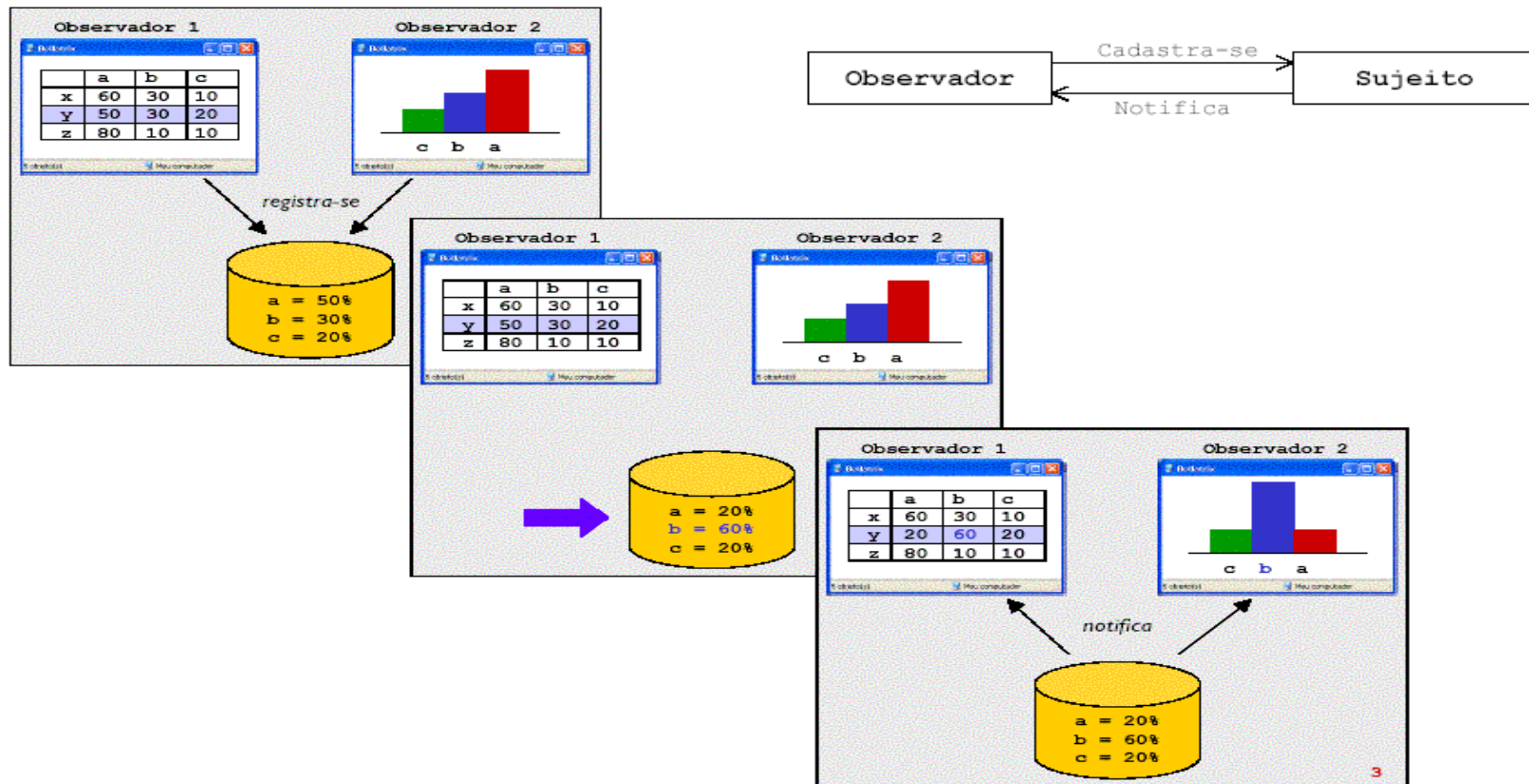
Observer

45

- Intenção: Definir uma **dependência um-para-muitos** entre objetos para que, quando um objeto X mudar de estado, objetos dependentes de X sejam notificados e automaticamente atualizados.
- ▣ GoF Book: *“Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.”*

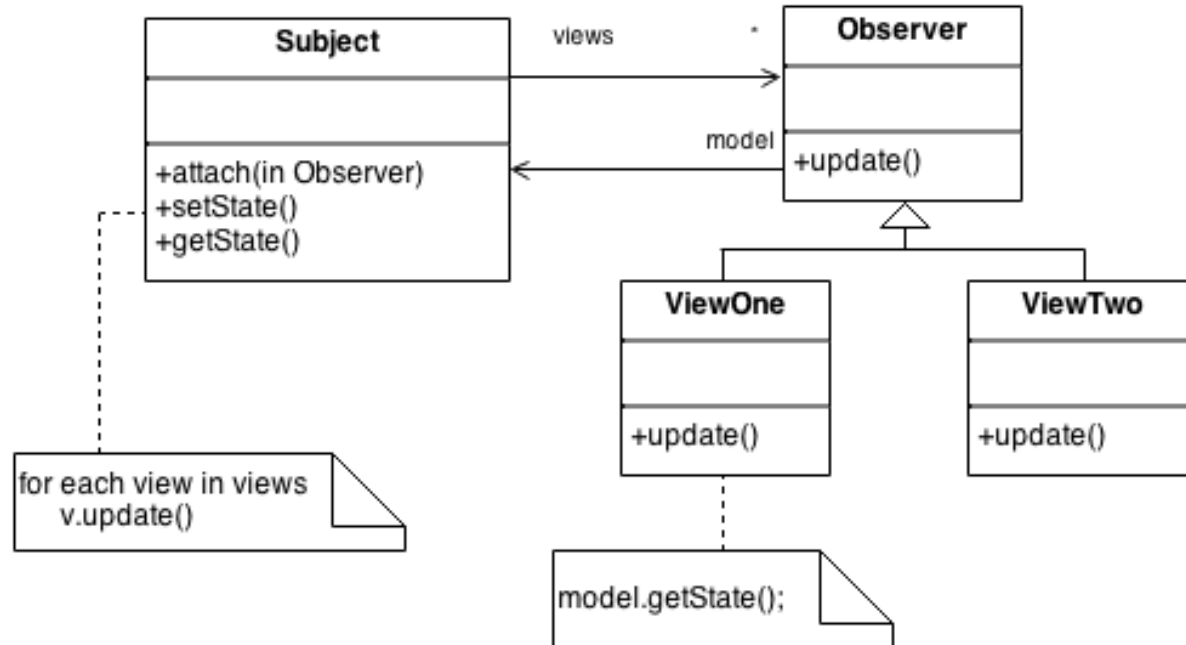
Observer (motivação)

46



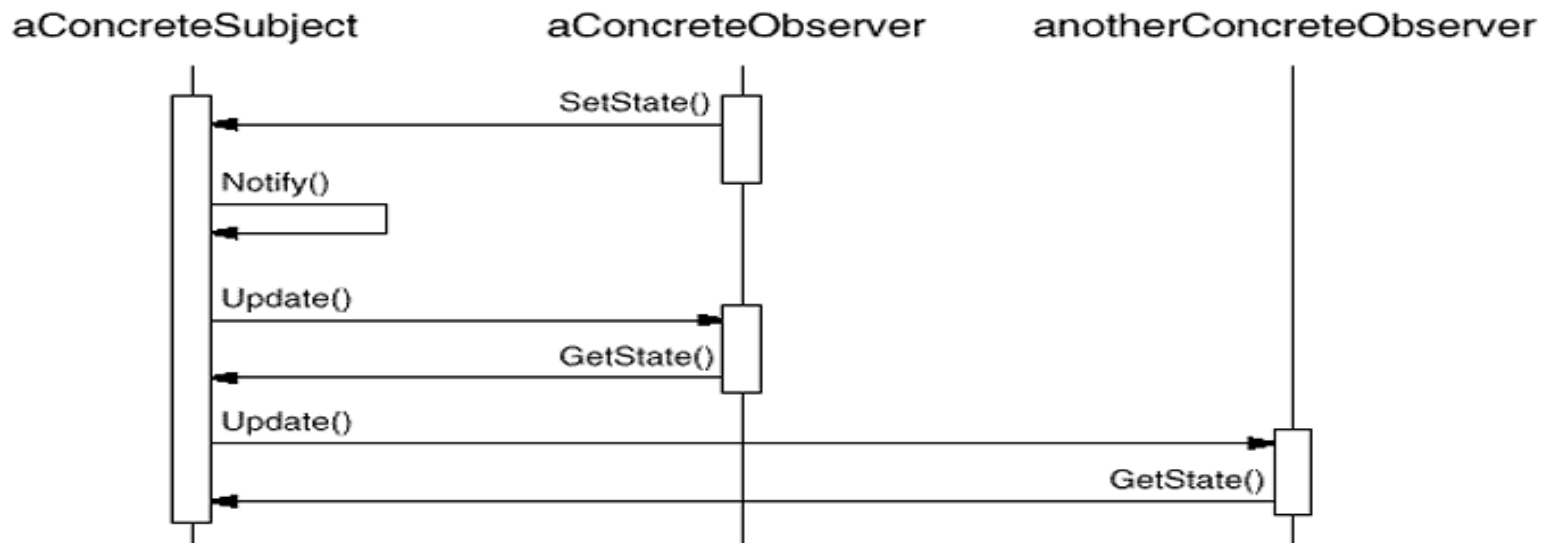
Observer - estrutura

47



Observer - esquema de interação

48



Observer – Java

```
public class ConcreteObserver
    implements Observer {

    public void update(Observable o) {
        ObservableData data = (ObservableData) o;
        data.getData();
    }
}
```

```
public class Observable {
    Observer observers = new ArrayList();

    public void add(Observer o) {
        observers.add(o);
    }

    public void remove(Observer o) {
        observers.remove(o);
    }

    public void notify() {
        Iterator it = observers.iterator();
        while(it.hasNext()) {
            Observer o = (Observer)it.next();
            o.update(this);
        }
    }
}
```

```
public class ObservableData
    extends Observable {
    private Object myData;

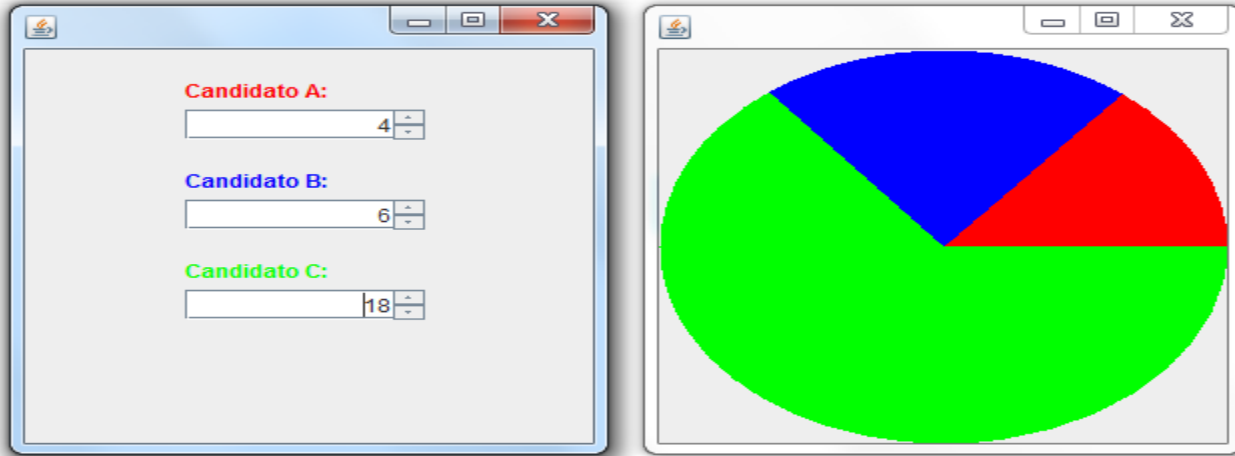
    public void setData(Object myData) {
        this.myData = myData;
        notify();
    }

    public Object getData() {
        return myData();
    }
}
```

```
public interface Observer {
    public void update(Observable o);
}
```

Observer – exemplo

50



Observer – exemplo (cont.)

51

```
14 public class ResultadoVotacao extends Observable {
15
16     private int qtdVotosA, qtdVotosB, qtdVotosC;
17
18     public ResultadoVotacao() {
19         qtdVotosA = qtdVotosB = qtdVotosC = 0;
20     }
21
22     public int getQtdVotosA() {
23
24     }
25
26     public void setQtdVotos1(int qtdVotos1) {
27         this.qtdVotosA = qtdVotos1;
28         setChanged();
29         notifyObservers();
30     }
31
32     public int getQtdVotosB() {
33
34     }
35
36     public void setQtdVotos2(int qtdVotos2) {
37         this.qtdVotosB = qtdVotos2;
38         setChanged();
39         notifyObservers();
40     }
41
42     public int getQtdVotosC() {
43         return qtdVotosC;
44     }
45
46     public void setQtdVotos3(int qtdVotos3) {
47         this.qtdVotosC = qtdVotos3;
48         setChanged();
49         notifyObservers();
50     }
51 }
```

Observer – exemplo (cont.)

52

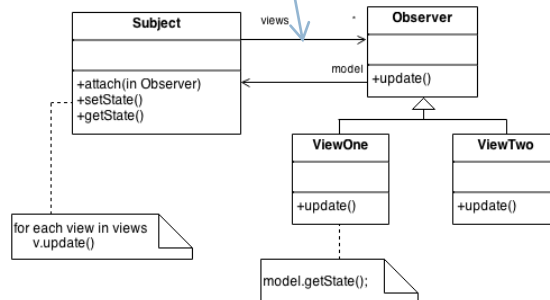
```
23 public class JPerspectivaGraficaVotacao extends JFrame implements Observer {
24
25     GraficoPizza graf;
26
27     JPerspectivaGraficaVotacao(ResultadoVotacao v) {
28         graf = new GraficoPizza();
29         getContentPane().add(graf);
30         setBounds(400, 300, 300, 300);
31     }
32
33     public void update(Observable o, Object arg) {
34         ResultadoVotacao v = (ResultadoVotacao) o;
35         int[] valores = new int[3];
36         valores[0] = v.getQtdVotosA();
37         valores[1] = v.getQtdVotosB();
38         valores[2] = v.getQtdVotosC();
39         graf.setFatias(valores);
40         repaint();
41     }
42 }
```

O método **update** é invocado sempre que o objeto observado é modificado.

Observer – exemplo (cont.)

53

```
5 public ResultadoVotacao v;  
6 public JPerspectivaGraficaVotacao observer1;  
7 public JPerspectivaTextualVotacao observer2;  
8  
9 public static void main(String[] args) {  
10     ResultadoVotacao v = new ResultadoVotacao();  
11  
12     JPerspectivaGraficaVotacao observer1 = new JPerspectivaGraficaVotacao(v);  
13     observer1.setVisible(true);  
14  
15     JPerspectivaTextualVotacao observer2 = new JPerspectivaTextualVotacao(v);  
16     observer2.setVisible(true);  
17  
18     v.addObserver(observer1);  
19     v.addObserver(observer2);  
20 }  
21 }
```



54

Memento

Memento

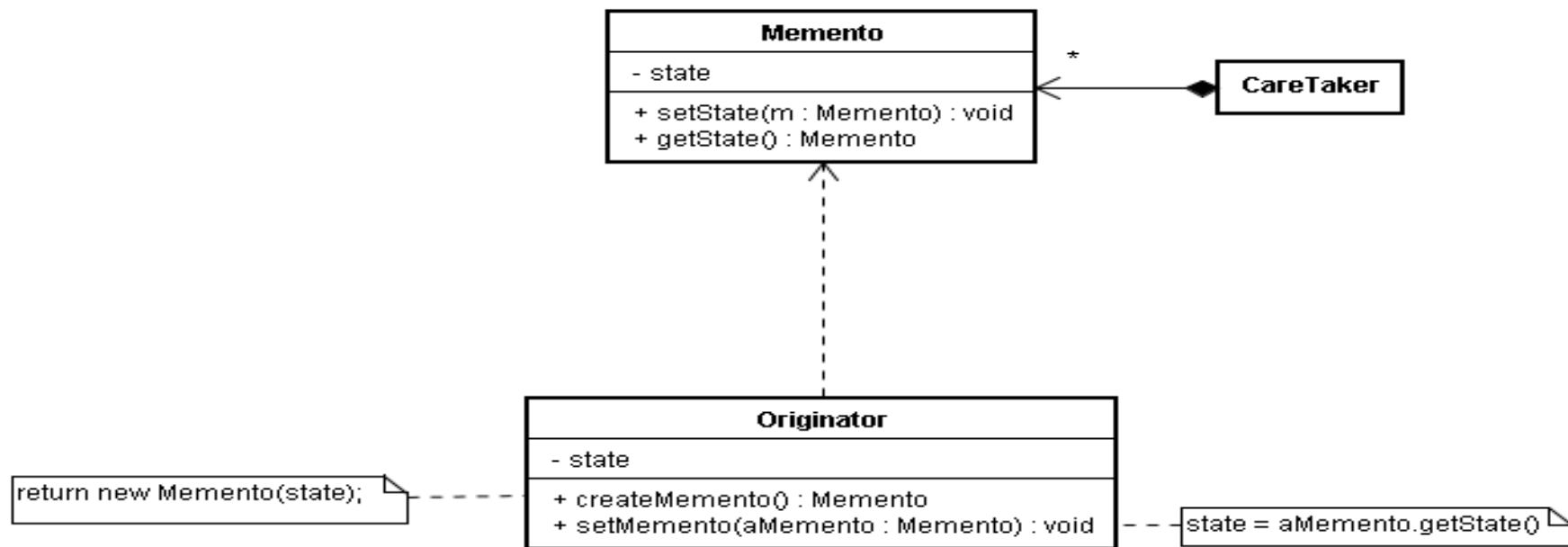
55

- Intenção: Sem violar o encapsulamento, capturar e externalizar o estado interno de um objeto de maneira que este estado possa ser utilizado novamente mais tarde para “restaurar” o objeto a um estado prévio.
- ▣ *GoF Book: “Without violating encapsulation, capture and externalize an object's internal state so that the object can be restored to this state later.”*

Memento permite implementar o conceito de “check points” sobre os estados de um objeto. Possibilita restaurar um objeto de volta a um estado anterior (e.g., operações de “undo” ou “roolback”).

Memento (estrutura)

56



Memento (Exemplos)

57

<https://imasters.com.br/back-end/padrao-de-projetos-com-memento>

<https://www.codeflow.site/pt/article/java-memento-design-pattern#example>

58

Factory Method

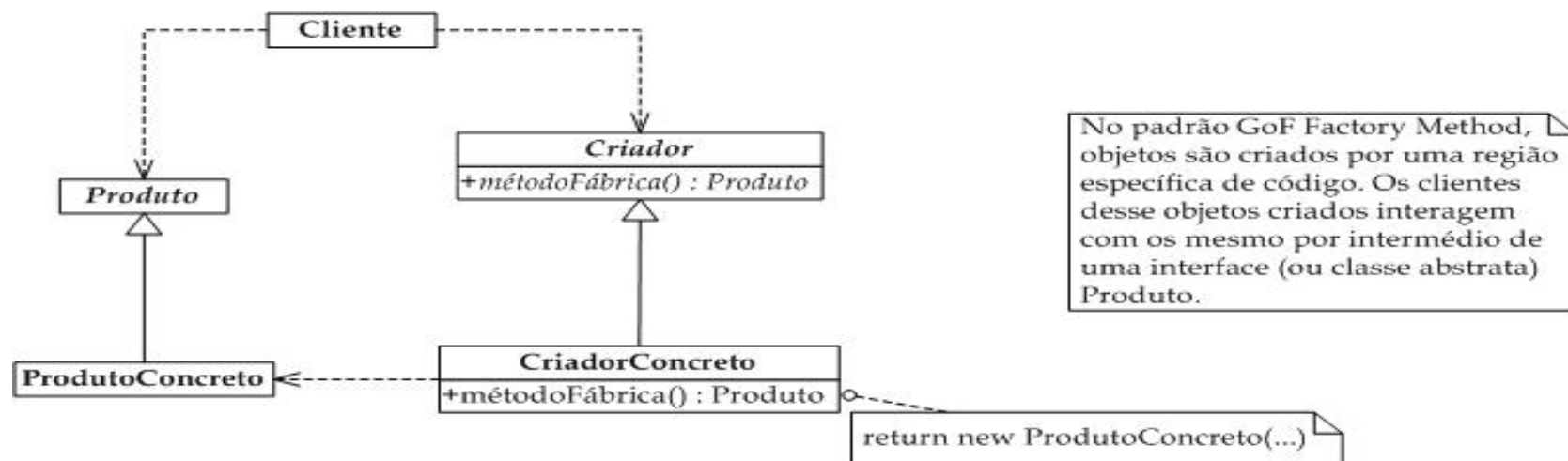
Factory Method

59

- Intenção: Definir uma interface para criação de um objeto, permitindo que subclasses decidam qual classe instanciar. Esse padrão permite a uma classe delegar para suas subclasses a responsabilidade de criação do produto.
 - GoF Book: *“Define an interface for creating an object, but let subclasses decide which class to instantiate. Factory Method lets a class defer instantiation to subclasses.”*

Factory Method - estrutura

60



Factory Method – exemplo 1

61

<https://www.youtube.com/watch?v=clyzpjIFDL4>

Factory Method – exemplo 2

62

```
5 public interface FuncionarioDao {  
6     void incluir(Funcionario f) throws NotFoundException;  
7     void alterar(Funcionario f) throws NotFoundException;  
8     List<Funcionario> getFuncionarios() throws NotFoundException;  
9 }
```

```
11 public class FuncionarioDaoJdbc implements FuncionarioDao {  
12  
13     private Connection c = null;  
14  
15+     FuncionarioDaoJdbc() throws SQLException {  
16  
17  
18  
19+     public void incluir(Funcionario f) {  
20  
21  
22  
23  
24  
25  
26  
27  
28  
29  
30  
31  
32+     public void alterar(Funcionario f) throws NotFoundException {  
33  
34  
35  
36  
37  
38  
39  
40  
41  
42  
43  
44  
45  
46  
47  
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
61  
62  
63  
64  
65  
66  
67  
68  
69  
70  
71  
72  
73  
74  
75  
76  
77  
78  
79  
80  
81  
82  
83  
84  
85  
86  
87  
88  
89+     public List<Funcionario> getFuncionarios() {  
90  
91  
92  
93  
94  
95  
96  
97  
98  
99  
100  
101  
102  
103  
104  
105  
106  
107  
108  
109  
110  
111  
112  
113  
114  
115  
116  
117  
118  
119  
120  
121  
122  
123  
124  
125  
126  
127  
128  
129  
130  
131  
132+     protected int databaseUpdate(Connection conn, PreparedStatement stmt)  
133  
134  
135  
136  
137  
138  
139  
140 }  
141 }
```

Factory Method – exemplo 2

63

```
3 public interface FuncionarioDaoFactory {  
4     public FuncionarioDao createFuncionarioDao();  
5 }
```

```
3 import java.sql.SQLException;  
4  
5 public class FuncionarioDaoFactoryJDBC implements FuncionarioDaoFactory {  
6  
7     private static FuncionarioDaoFactoryJDBC instance = null;  
8  
9     public static FuncionarioDaoFactoryJDBC getInstance() {  
10         if (instance == null)  
11             FuncionarioDaoFactoryJDBC.instance = new FuncionarioDaoFactoryJDBC();  
12         return instance;  
13     }  
14  
15     public FuncionarioDao createFuncionarioDao() {  
16         try {  
17             return new FuncionarioDaoJdbc();  
18         } catch (SQLException e) {  
19             System.out.println("Não foi possível gerar o objeto");  
20         }  
21         return null;  
22     }  
23 }
```

Factory Method - exemplo

64

```
8 public class Main {
9
10     public static void main(String[] args) throws NotFoundException,
11         InstantiationException, IllegalAccessException {
12         try {
13             SimpleDataSource.init("bd.properties");
14
15             FuncionarioDao dao = SimpleDataSource.getFabrica()
16                 .createFuncionarioDao();
17
18             Funcionario f = new Funcionario();
19             f.setID(12);
20             f.setDatacontratacao("12/12/2006");
21             f.setMatricula(12345);
22             f.setNome("João Paulo");
23             dao.incluir(f);
24
25             List<Funcionario> funcionarios = dao.getFuncionarios();
26             Iterator<Funcionario> iter = funcionarios.iterator();
27             while (iter.hasNext()) {
28                 f = iter.next();
29                 System.out.println(f.getMatricula() + "\t" + f.getNome());
30             }
31         } catch (SQLException ex) {
32             ex.printStackTrace();
33         } catch (ClassNotFoundException ex) {
34             ex.printStackTrace();
35         } catch (IOException ex) {
36             ex.printStackTrace();
37         }
38     }
```

Factory Method - exemplo

65

```
5 public class FuncionarioDaoJPA implements FuncionarioDao {
6
7+   FuncionarioDaoJPA() {}
8
9
10+  public void incluir(Funcionario f) {}
11
12
13+  public void alterar(Funcionario f) throws NotFoundException {}
14
15
16+  public List<Funcionario> getFuncionarios() {}
17
18
19
20 }
```

```
3 public class FuncionarioDaoFactoryJPA implements FuncionarioDaoFactory {
4
5     private static FuncionarioDaoFactoryJPA instance = null;
6
7-   public static FuncionarioDaoFactoryJPA getInstance() {
8       if (instance == null)
9           FuncionarioDaoFactoryJPA.instance = new FuncionarioDaoFactoryJPA();
10      return instance;
11  }
12
13-   public FuncionarioDao createFuncionarioDao() {
14       return new FuncionarioDaoJPA();
15  }
```

66

Abstract Factory

Abstract Factory

67

- Intenção: Fornecer uma interface comum para a criação de **famílias de objetos relacionados ou dependentes**, sem especificar suas classes concretas.
- ▣ GoF Book: *“Provide an interface for creating families of related or dependent objects without specifying their concrete classes.”*

Abstract Factory

68

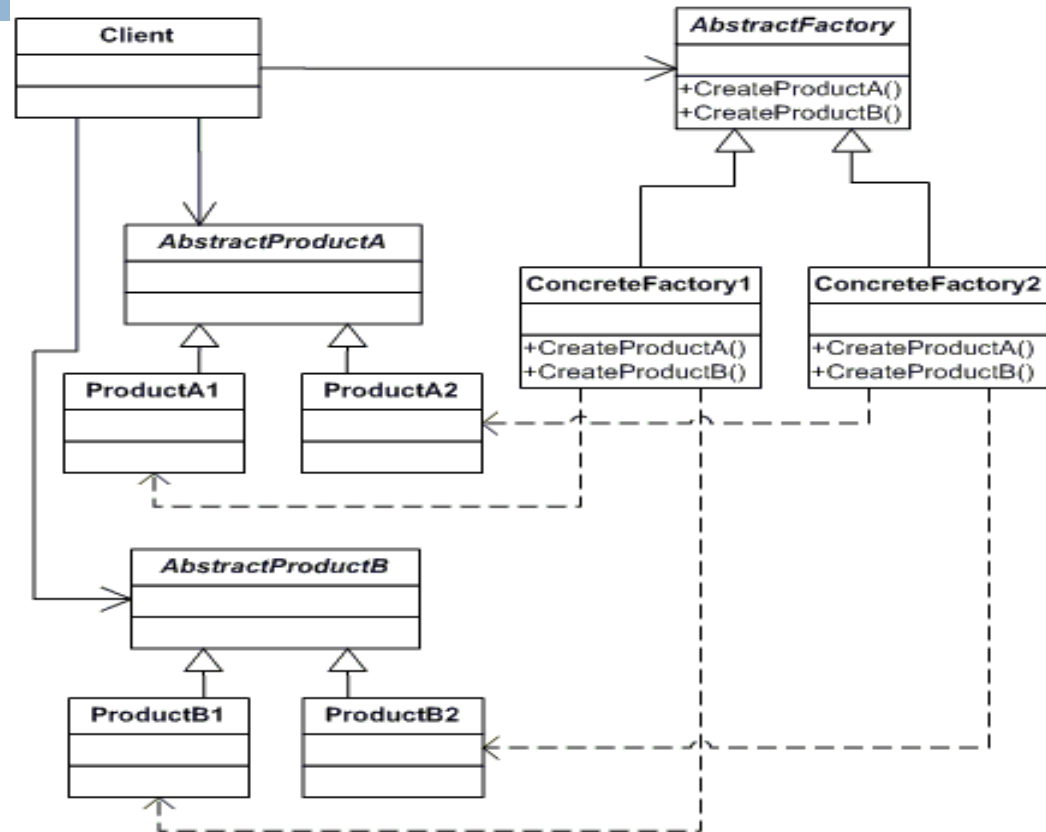
□ Estrutura

- ▣ Há duas hierarquias de classes paralelas a de produtos e a de fábricas dos produtos.
- ▣ Há uma interface que representa uma fábrica para cada família de objetos.
- ▣ As subclasses concretas na hierarquia de fábricas instanciam cada família específica contida na hierarquia de produtos.

Abstract Factory

69

□ Estrutura



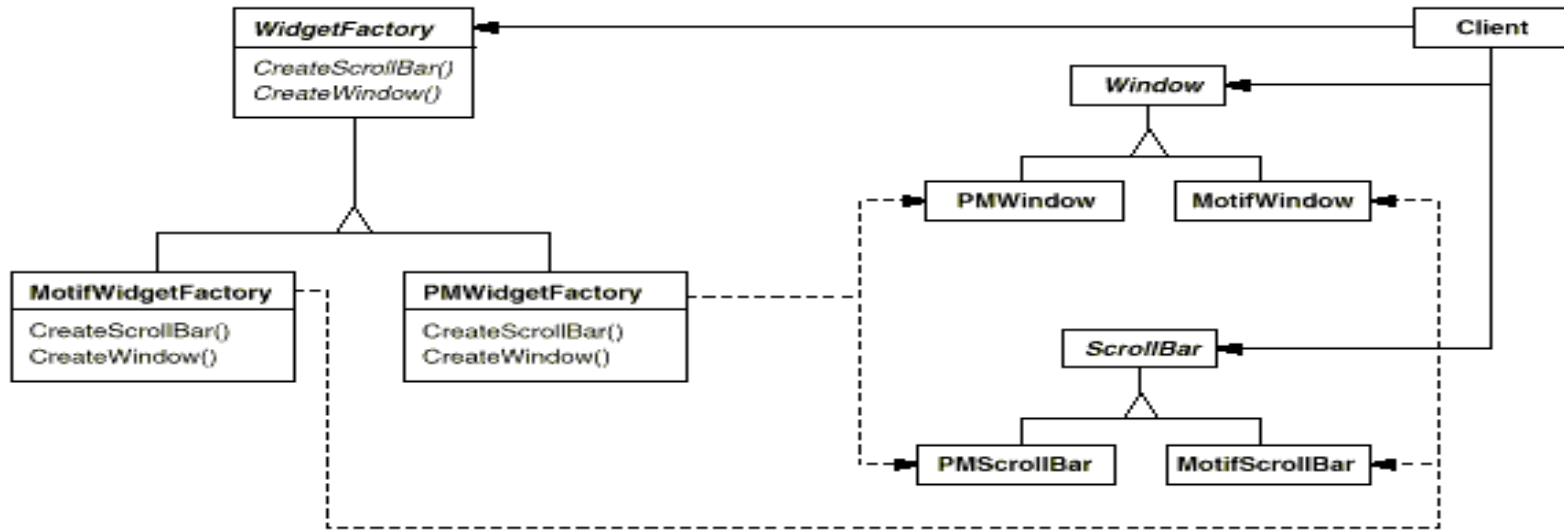
Abstract Factory - exemplo

70

- No sistemas UNIX, há várias bibliotecas de IU: Gnome, KDE, etc.
- Cada biblioteca fornece componentes da IU (botões, menus suspensos, ...) com aparências diferentes.
- De que forma usar AbstractFactory para permitir a criação de aplicações portáteis (i.e., que funcionam em Gnome, KDE, etc.)?

Abstract Factory – exemplo (cont.)

71



Referências

72

- Professor Prof. Eduardo Bezerra.
- Gamma et al. ***Padrões de Projeto - Soluções Reutilizáveis de Software Orientado a Objetos***, Porto Alegre: Bookman, 2000.
- Steven Metsker, ***Padrões de Projeto em Java***, Porto Alegre: Bookman, 2004.
- Freeman et al. ***Use a Cabeça!: Padrões de Projetos***, Rio de Janeiro: Alta Books, 2005.

Referências

73

